RuleShift-Web: A Replayable Workbench for Auditing Agent Memory under Changing Web Policies

Chih-Kai Wang
National Taipei University of Education
Taipei, Taiwan

Abstract

Web agents can retain a policy response that is still fresh under its HTTP cache lifetime even after a site begins enforcing a revised rule. A successful revalidation can also become obsolete before a consequential transaction commits. RuleShift-Web is a local research workbench that makes these two boundaries observable and replayable. It publishes versioned retail policies as real HTTP resources, records source-linked agent evidence, enforces an application-level policy revision check at transaction commit, and exposes the sequence of responses, publications, attempted actions, and state changes in an interactive trace. A frozen study evaluates five evidence strategies on 3,200 executions over paired tasks from two open models, plus 1,600 decisions from a deterministic controller. In this one-relevant-source harness, source-dependent validation matches all-source conditional validation while avoiding four policy reads per task; unconditional refresh has higher observed accuracy for both models. The workbench therefore exposes the cost and limits of memory validation rather than claiming a universally superior controller. Visitors can reproduce policy changes and inspect saved development-model traces without a conference network connection.

CCS Concepts

• **Information systems** → **Web applications**; • **Computing methodologies** → *Intelligent agents.*

Keywords

Web agents, agent memory, HTTP caching, dynamic policies, reproducible evaluation

ACM Reference Format:
Chih-Kai Wang. 2027. RuleShift-Web: A Replayable Workbench for Auditing Agent Memory under Changing Web Policies. In *Proceedings of The Web Conference 2027 (WWW '27).* ACM, New York, NY, USA, 3 pages.

1 Motivation and scope

An agent may read a return policy at time t_1, carry a derived action to t_2, and encounter a policy publication between those times. Three objects then diverge: the response held by the client, the representation selected by the HTTP origin, and the policy enforced at transaction commit. A cache-fresh response is not necessarily current in the site's business process; an HTTP `304 Not Modified` validates a selected representation at the time of that request, not an action performed later [5]. These distinctions are difficult to inspect when a benchmark collapses all policy access into an abstract tool call.

Table 1: Prior evaluation emphasis versus the interactive Web probe.

Work	Main inspectable object
STALE [2]	Implicit conflict and revised belief
EvoArena [1]	Memory evolution across changing environments
EnvTrust [4]	Environmental evidence and agent outcome
PlanFence [3]	Plan lineage and action-time owner validation
This demo	HTTP representation, held cache evidence, and separate commit authority

Prior work already studies obsolete memory and downstream behavior [2], memory evolution in changing environments [1], dependency-scoped validation before external actions [3], and agent grounding in environmental evidence [4]. RuleShift-Web does not claim to invent stale-memory detection, dependency-scoped validation, conditional HTTP requests, or transaction revision checks. Its candidate contribution is an experimental instrument in which visitors can schedule policy publication, acquire or reuse evidence, and submit a transaction while inspecting a fixed commit-time guard and its event trace. PlanFence already checks declared action dependencies against their owners [3]; here the visible objects are Web responses and validators, cache age, and a separate application revision check. Their separation is the workbench's subject, not a new validation protocol. Table 1 positions that narrower probe against the cited studies.

2 System design

2.1 A policy site with separate authority surfaces

The prototype runs a FastAPI origin on a local host. Policies for discounts, shipping, returns, fulfillment, and address changes are served as HTTP resources with ETags and a 120-second cache lifetime. An unchanged conditional GET returns `304`; a relevant publication returns a new representation and validator. A publication of an unrelated policy leaves the relevant validator unchanged. A `200` alone does not prove that a representation changed. Reverting A→B→A restores the representation's original validator. The application-level revision is also content-derived: after this reversion, an old A revision can commit an action that is valid under current A. The workbench tests current-rule consistency, not whether any intervening publication occurred.

WWW '27, Dublin, Ireland
2027.

Agent-facing endpoints expose public policy, order inspection, and retail operations. Researcher endpoints publish rules and expose traces; they never enter agent prompts. Each transaction carries an expected policy revision. The origin compares it with the active rule under the same lock as the state mutation. A mismatch returns `409` without mutation. This application-level guard is distinct from HTTP `If-Match` on the policy resource. Eligibility is checked after the revision match, so the trace separates stale attempts from current but prohibited actions by their response error fields; both can have status `409`.

2.2 Evidence and replay

An evidence record retains its source URL, acquisition time, validator, content revision, policy body, dependent source, and last successful validation time. The controller can reuse a remembered response, refresh it unconditionally, validate with `If-None-Match`, or check only sources relevant to the pending action. A transport or HTTP error leaves verification unresolved; it is never treated as a successful validation. If no HTTP validator is available, a full GET and content comparison replace conditional revalidation. The prompt builder passes only the public task and evidence to a model. Research snapshots, future publications, and scoring data remain outside model input.

The workbench records each policy read, publication, order inspection, and transaction attempt with its response and resulting state hash. A verifier reconstructs a run from a clean initial state and checks the resulting events and state. The local web page lets a visitor publish policies, choose an evidence check, submit an operation, and inspect the timeline. It uses an application-managed evidence cache; explicit network checks bypass browser cache storage so each intended request reaches the origin. A static bundle contains a 41-event example and seven recorded purchase interventions, so trace inspection and branch selection do not require conference Wi-Fi. The offline branches display previously verified deterministic event records. A separately labelled set of eight local Qwen development traces exposes raw model output, evidence history, rejections, and final state without generating fresh model decisions.

3 Hands-on demonstration

A visitor starts with policy P1 permitting a discount and reads its HTTP representation. The visitor then publishes P2, which removes eligibility, while leaving the client-held P1 response in place. Reusing that response produces an attempted transaction with the old revision; the origin rejects it without mutation. A conditional GET instead receives P2, after which the visitor can run a non-LLM public-evidence check that denies without sending a transaction. This interface does not generate a live model decision; saved model decisions are inspected separately. An unconditional reload also uses one GET; when unchanged it transfers a body rather than returning `304`. Publishing an unrelated shipping rule demonstrates that a discount validator remains unchanged.

In a second branch, the visitor validates a permissive policy, then publishes a restrictive version just before commit. The transaction is again rejected under the active revision. After a bounded refresh, the controller can reach the current decision. Returning to P1 demonstrates that identical policy content can regain the same validator. The interface shows the held evidence, request and response, transaction result, committed state, and replay record. The same origin provides return, shipment, and address-change examples. For a conference venue, the intended deployment is a laptop-bound local service with a static trace fallback and an interactive poster walkthrough. A read-only table in both modes compares the frozen five-strategy results; it is labelled separately from live user interventions and saved model development traces. A prepared supplementary review bundle contains the offline page, verified traces, local-origin code, manuscript, and audit summaries.

Table 2: Correct final decision and state, out of 320 tasks per cell.

Strategy	Qwen	Phi	Rule controller
Retrieval	273	141	300
Prior selective	274	141	300
Always refresh	**287**	**223**	**320**
Conditional all	284	195	**320**
Source dependent	284	195	**320**

4 Evaluation and limitations

4.1 Frozen protocol

We generated four held-out change scenarios: no change, relevant policy change, unrelated policy change, and A→B→A reversion. Each has ten distinct eight-task streams; the same 320 task positions are reused across strategies and models. The five compared strategies are memory retrieval without proactive validation, unconditional refresh of the relevant source, conditional validation of all five sources, a prior selective-retry controller, and the proposed source-dependent conditional validation. Each task declares one relevant policy source; conditional-all checks five sources but passes the same relevant evidence to the model. Initial site state, public observations, tasks, and action budget are paired across strategies. An in-process FastAPI TestClient drives the origin, with scheduled publications before each task's evidence check. The evaluated models are Qwen3-4B-Instruct-2507 and Phi-4-mini-instruct at pinned revisions, using greedy float16 CUDA inference on one NVIDIA L4. An exact Markdown-fence adapter for Phi was selected on development data before the held-out protocol was frozen; Qwen uses a strict parser. A deterministic public-evidence controller runs the same streams as a strong non-LLM reference.

The primary outcome requires a correct final decision *and* matching transaction state. We preserve wrong actions, rejected transactions, raw model output, response-contract failures, HTTP requests, token counts, source evidence, and replayable origin events. The audit independently reconstructs the world and checks every result against the frozen stream schedule. Paired accuracy differences use 40 eight-task streams, stratified by scenario, with 10,000 bootstrap draws and two-sided 95% percentile intervals. The 3,200 model executions and 1,600 separate control decisions completed with no failed arms. The common server guard prevented invalid commits; this does not imply that the agents avoided invalid attempts.

Table 3: Total agent HTTP requests over 320 tasks per cell, including actions.

Strategy	Qwen	Phi
Retrieval	737	555
Prior selective	794	555
Always refresh	1,017	890
Conditional all	2,300	2,149
Source dependent	1,020	869

4.2 What the traces show

Source-dependent validation ties all-source conditional validation in accuracy for both models. Conditional-all makes 1,800 policy reads per model, versus 520 for source-dependent validation: the saving of four reads per task follows from checking one declared source instead of five. Total requests also include action attempts (Table 3); they are not a cost-adjusted success metric. Relative to retrieval, source-dependent validation's paired accuracy gain is 3.44 percentage points for Qwen (95% interval 1.56–5.31) and 16.88 for Phi (11.88–22.19). Always refresh has higher observed accuracy: 287 versus 284 for Qwen and 223 versus 195 for Phi. The source-minus-always intervals are $[-1.88, 0.00]$ and $[-16.88, -2.50]$ percentage points, respectively; Qwen's interval includes zero. Source-dependent and always-refresh each make 520 policy reads. Phi's 21 fewer total requests under source-dependent validation correspond to 21 fewer attempted transactions, with lower completion. Request count alone does not establish greater efficiency.

The no-LLM controller scores 320/320 with all three proactive validation strategies and 300/320 with retrieval or prior selective retry. This separates controller mistakes from avoidable memory staleness. Qwen's three source-dependent errors relative to always refresh are address-age judgments with identical policy body and revision but different evidence acquisition timestamps. The model's prompts and decisions are retained; these differences do not show an incorrect HTTP validator. Phi produces 1,626 raw parser rejections, many due to Markdown fences. After the predeclared adapter, 685 *response-level* contract failures remain, including retry responses; all have contradictory `attempt` and `reason` fields, not malformed JSON. In proactive Phi arms, always-refresh has 97 final contract rejections versus 125 for source-dependent validation; no accepted final decision is incorrect in those arms. Thus the observed gap characterizes this configured model–prompt–parser pipeline, not a demonstrated freshness advantage. The adapter asymmetry also prevents a pure cross-model quality ranking.

The live origin was tested for unchanged `304`, changed `200`, stale-commit `409`, irrelevant publication, reversion, commit-time races, and source failures. Saved traces can be re-executed against a clean initial state, checking HTTP responses and final state hashes; the offline page only loads the previously verified records. Seven live-origin sequences were previously reported as passing. In an additional Safari run, the exported three-event trace (read `200`, publish `200`, purchase `409`) was independently replayed with an unchanged final state. These live-socket and browser checks are separate from the in-process formal matrix; that matrix does not test a publication between validation and commit. The study uses binary decisions, declared single-source dependencies, fixed publication schedules, a synthetic retail site, and two small models. Its intervals describe generated-stream variation rather than reliability on real commerce websites. The local conference deployment must still be checked on the presentation laptop.

5 Ethical use of data

The prototype uses synthetic policies and orders. It involves no human participants, personal customer records, payment credentials, or real transactions. Evaluated agents are confined to the controlled site routes; researcher controls and scoring records are not agent observations. The interactive service is intended to run locally rather than expose an unauthenticated public endpoint. Generative AI tools assisted with implementation, analysis, and manuscript drafting. The sole human author is responsible for verifying the reported evidence, citations, and claims before submission.

References

[1] J. Xu et al. 2026. EvoArena: Tracking Memory Evolution for Robust LLM Agents in Dynamic Environments. *arXiv:2606.13681*. https://arxiv.org/abs/2606.13681.
[2] H. Chao, Y. Bai, R. Sheng, T. Li, and Y. Sun. 2026. STALE: Can LLM Agents Know When Their Memories Are No Longer Valid? *arXiv:2605.06527*. https://arxiv.org/abs/2605.06527.
[3] E. Chen, S. Wang, and C. G. Brinton. 2026. Fresh Memory, Stale Plans: Dependency-Scoped Validation for Distributed LLM-Agent Memory. *arXiv:2609.03340*. https://arxiv.org/abs/2609.03340.
[4] S. Sheng, Z. Wang, and L. Zhou. 2026. When Agents Overtrust Environmental Evidence: An Extensible Agentic Framework for Benchmarking Evidence-Grounding Defects in LLM Agents. *arXiv:2605.08828*. https://arxiv.org/abs/2605.08828.
[5] R. Fielding, M. Nottingham, and J. Reschke. 2022. HTTP Semantics. RFC 9110. https://www.rfc-editor.org/rfc/rfc9110.html.